

Helix OTA — As-Built Operational & Rollout REST Endpoints (/api/v1)

Field	Value
Revision	1
Last modified	2026-06-08T00:00:00Z
Created	2026-06-08
Status	active (as-built — generated from the real route table) An as-built reference for the operational + staged-rollout REST endpoints actually wired in <code>server/internal/api/server.go</code> (the audit read, telemetry reads, device-group CRUD + membership, and the staged-rollout create/get/evaluate routes). It documents only routes that exist in the code and cites the handler
Status summary	<code>file:func</code> each path maps to. It is the implemented counterpart to the prose specs operational_endpoints.md (audit/telemetry/groups) and 1.0.1-staged-rollout/rollback_ux.md (rollback); where the implementation and the spec disagree, the divergence is called out in §10. The operator recall endpoint from ../1.0.1-staged-rollout/rollback_ux.md §7 is marked PLANNED / not-yet-wired.
Generated from	<code>server/internal/api/server.go</code> route table — <code>Router()</code>, lines 104–167 (specifically the protected auth group, lines 122–164, plus the unversioned probes on lines 111–112). This document was hand-authored from a direct read of that route table and the handler sources; it is not auto-generated, but every row traces to a cited line/function.
Anti-bluff	Per Constitution §11.4.6 (no-guessing): only routes present in <code>server.go</code> are documented. Status/Action/Reason enum values are taken from the <code>ota-</code>

Field	Value
	rollout-engine brick source (types.go, verdict.go), not invented. Spec/impl divergences are listed in §10, never silently reconciled.
Owner	Helix OTA control-plane team
Related	endpoints.md (parent REST spec — conventions, error model, RBAC); operational_endpoints.md (audit/telemetry/groups prose spec); openapi.yaml ; ../1.0.1-staged-rollout/rollout_engine.md ; ../1.0.1-staged-rollout/rollback_ux.md

table_of_contents

1. [purpose_and_scope](#)
2. [how_to_read_this_document](#)
3. [as_built_endpoint_table](#)
4. [audit_read_endpoint](#)
5. [telemetry_read_endpoints](#)
6. [device_group_crud_and_membership](#)
7. [staged_rollout_endpoints](#)
8. [planned_recall_endpoint_not_yet_wired](#)
9. [audit_write_middleware_cross_cutting](#)
10. [spec_vs_implementation_divergences](#)

1. purpose_and_scope

This is the **as-built** reference for the operational and staged-rollout endpoints wired this session into the Helix OTA control plane. It documents, for each route that **actually exists in server/internal/api/server.go**:

- HTTP method + path (relative to the /api/v1 base);
- required roles (from the literal requireRole(...) call on that route in server.go);
- request body type and response type + key fields (from the handler + wire-type sources);
- status codes the handler can emit;
- the handler file:func the route maps to.

Conventions (base path /api/v1, the { "error": { code, message, request_id, details[] } } envelope, RBAC roles admin/operator/viewer/device, transport/compression, pagination header ?limit/?cursor) are **inherited from endpoints.md** and not re-specified here.

In scope (all confirmed present in the route table):

- **Audit read** — GET /audit (server.go:163).
- **Telemetry reads** — GET /devices/:deviceId/telemetry (server.go:148) and GET /telemetry/overview (server.go:149).
- **Device-group CRUD + membership** — POST/GET/GET/PATCH/DELETE /groups[...] and the members sub-routes (server.go:153–160).
- **Staged rollout** — POST/GET /deployments/:deploymentId/rollout and POST /deployments/:deploymentId/rollout/evaluate (server.go:140–142).

Out of scope: the auth, device-register, artifact, release, deployment-CRUD, and client (device) endpoints — those are the 1.0.0-MVP base surface documented in [endpoints.md](#). The operator **recall** endpoint is **not** in the route table and is documented as PLANNED in §8.

2. how_to_read_this_document

Each endpoint section states the **verified** facts pulled from source:

- **Roles** are the exact arguments to `requireRole(...)` in `server.go` — the codebase expresses the role hierarchy by listing all accepted roles (e.g. `requireRole(RoleViewer, RoleOperator, RoleAdmin)`), so `admin` $\supseteq$ `operator` $\supseteq$ `viewer` is realized by enumeration, not inheritance.
- **Request / Response types** name the Go wire structs (and their JSON field tags) in the handler files; the JSON examples below are rendered from those struct tags.
- **Maps to** gives `relative/handler.go:HandlerFunc`.

Where the implemented shape differs from the prose spec it implements, the difference is noted inline and collected in §10.

3. as_built_endpoint_table

Method	Path (/api/v1 + ...)	Required roles	Request type	Response type
GET	/audit	admin	— (query params)	AuditLogList
GET	/devices/:deviceId/telemetry	viewer, operator, admin, device (own id)	—	TelemetryHistory
GET	/telemetry/overview	viewer, operator, admin	—	TelemetryOverview
POST	/groups	operator, admin	GroupCreate	GroupView
GET	/groups		—	GroupList

Method	Path (/api/v1 + ...)	Required roles	Request type	Response type
		viewer, operator, admin		
GET	/groups/:groupId	viewer, operator, admin	—	GroupView
PATCH	/groups/:groupId	operator, admin	GroupUpdate	GroupView
DELETE	/groups/:groupId	admin	—	— (204)
GET	/groups/:groupId/members	viewer, operator, admin	—	GroupMembers
POST	/groups/:groupId/members	operator, admin	MemberAdd	— (204)
DELETE	/groups/:groupId/ members/:deviceId	operator, admin	—	— (204)
POST	/deployments/:deploymentId/ rollout	operator, admin	RolloutCreate	RolloutState
GET	/deployments/:deploymentId/ rollout	viewer, operator, admin	—	RolloutState
POST	/deployments/:deploymentId/ rollout/evaluate	operator, admin	RolloutVerdict	RolloutDecision
POST	/deployments/ {deploymentId}/recall	—	—	—
GET	/deployments/ {deploymentId}/rollbacks	—	—	—

The path param uses the gin :name form in the route table; the public-facing spec uses {name}. They are the same parameter.

4. audit_read_endpoint

4.1 GET /api/v1/audit

Reads the audit trail. Admin-only.

- **Roles:** admin (requireRole(RoleAdmin), server.go:163).
- **Maps to:** handlers_audit.go:handleListAudit.
- **Request:** no body. Query parameters (all optional, combined as AND):
 - ?action= — exact action verb filter.
 - ?resource_type= — exact resource-type filter.
 - ?cursor= — opaque pagination cursor.

- ?limit= — integer in [1, 200] (default 50). Out of range → 400
VALIDATION_FAILED with details: [{field:"limit", issue:"out of range"}].

• **Response 200** (AuditLogList, audit_wire.go):

```
{
  "items": [
    {
      "id": "9c2f...",
      "actor": "admin@example.com",
      "action": "DEPLOYMENT_CREATE",
      "resource_type": "deployment",
      "resource_id": "d12b...",
      "details": { "version": "1.1.0" },
      "ip_address": "203.0.113.7",
      "user_agent": "helix-dashboard/1.0",
      "created_at": "2026-06-08T10:15:00Z"
    }
  ],
  "next_cursor": null
}
```

- **Key fields** (AuditLogEntry): id, actor (a **flat string** — the actor subject, falling back to the user id; audit_wire.go:toAuditLogEntry), action, resource_type, resource_id (omitempty), details (string→string map, oitempty), ip_address (omitempty), user_agent (omitempty), created_at. next_cursor is *string (JSON null when exhausted).
- **Status codes**: 200 OK; 400 VALIDATION_FAILED (bad limit); 500 INTERNAL (store error → "could not list audit log"). Auth failures (401/403) are produced by the shared authMiddleware + requireRole ahead of the handler.
- **Immutability**: read-only over the API — there is no POST/PATCH/DELETE / audit. Rows are written only by the audit middleware (§9).

5. telemetry_read_endpoints

5.1 GET /api/v1/devices/{deviceId}/telemetry

Per-device telemetry event history.

- **Roles**: viewer, operator, admin, device (server.go:148). A device token may read **only its own** id: the handler compares claims.Subject to the path deviceId and returns 403 FORBIDDEN ("a device may read only its own telemetry") for a non-privileged caller reading another device.
- **Maps to**: handlers_telemetry.go:handleDeviceTelemetry.
- **Request**: no body, no query parameters consumed by the handler.
- **Response 200** (TelemetryHistory):

```
{
  "device_id": "8f3a...",
  "events": [
    {
```

```

    "event": "download_started",
    "version": "1.1.0",
    "deployment_id": "d12b...",
    "error_code": "",
    "detail": "",
    "timestamp": "2026-06-08T10:15:00Z",
    "received_at": "2026-06-08T10:15:01Z"
  }
]
}

```

- **Key fields** (TelemetryEventView): event (otaprotocol.TelemetryEvent), version, deployment_id, error_code, detail (all omit empty), timestamp, received_at. Events are returned in insertion order (the store contract TelemetryForDevice, store.go:179–181).
- **Status codes:** 200 OK; 403 FORBIDDEN (cross-device device token); 500 INTERNAL ("could not read telemetry").

5.2 GET /api/v1/telemetry/overview

Fleet-wide telemetry counts grouped by event type.

- **Roles:** viewer, operator, admin (server.go:149). No device access.
- **Maps to:** handlers_telemetry.go:handleTelemetryOverview.
- **Request:** no body, no query parameters.
- **Response 200** (TelemetryOverview):

```

{
  "event_counts": {
    "download_started": 1188,
    "installing": 1152,
    "success": 1100,
    "failure": 36
  },
  "total": 3476
}

```

- **Key fields** (TelemetryOverview): event_counts (string→int64 map, keyed by event type via the store TelemetryEventCounts, store.go:182–184) and total (sum of all counts, computed in the handler). There is **no** device-state breakdown, scope object, or failure_rate in the built response (see §10).
- **Status codes:** 200 OK; 500 INTERNAL ("could not aggregate telemetry").

6. device_group_crud_and_membership

Wire types: handlers_group.go. Store seam: store.go (CreateGroup, GetGroup, ListGroups, UpdateGroup, DeleteGroup, AddGroupMember, ListGroupMembers, RemoveGroupMember).

6.1 POST /api/v1/groups

- **Roles:** operator, admin (server.go:153). **Maps to:** handleCreateGroup.
- **Request body** (GroupCreate): { "name": "...", "description": "...". } name is **required** — blank → 400 VALIDATION_FAILED (details: [{field:"name",issue:"required"}]).
- **Response 201** (GroupView): { "id", "name", "description"(omitempty), "created_at" }.
- **Status codes:** 201 Created; 400 VALIDATION_FAILED (malformed body / blank name); 409 CONFLICT (store.ErrConflict → "a group with that name already exists"); 500 INTERNAL.

6.2 GET /api/v1/groups, GET /api/v1/groups/{groupId}

- **Roles:** viewer, operator, admin (server.go:154–155).
- **List** (handleListGroup): response GroupList { "items": [GroupView] } — **no pagination** (ListGroup takes no filter/cursor; 500 INTERNAL on store error).
- **Read one** (handleGetGroup): response GroupView; 404 NOT_FOUND ("group not found") if absent.

6.3 PATCH /api/v1/groups/{groupId}, DELETE /api/v1/groups/{groupId}

- **PATCH** (handleUpdateGroup, server.go:156): roles operator, admin. Body GroupUpdate { "name", "description" }. The handler loads the existing group (404 if absent), applies name only when non-empty, and **always** sets description from the body (a missing description clears it — see §10). Response 200 GroupView. 409 CONFLICT on name collision; 404 NOT_FOUND; 500 INTERNAL.
- **DELETE** (handleDeleteGroup, server.go:157): role admin only. Response 204 No Content. Any store error maps to 404 NOT_FOUND ("group not found").

6.4 membership endpoints

- **GET /groups/{groupId}/members** (handleListGroupMembers, server.go:158): roles viewer, operator, admin. Response GroupMembers { "group_id", "device_ids": [...] } (empty array, never null). 404 NOT_FOUND if the group is missing.
- **POST /groups/{groupId}/members** (handleAddGroupMember, server.go:159): roles operator, admin. Body MemberAdd { "device_id": "...". } — **a single device id**, required (blank → 400 VALIDATION_FAILED). Response 204 No Content. 404 NOT_FOUND (store.ErrNotFound → missing group); 500 INTERNAL.
- **DELETE /groups/{groupId}/members/{deviceId}** (handleRemoveGroupMember, server.go:160): roles operator, admin. Response 204 No Content. 404 NOT_FOUND (missing group); 500 INTERNAL.

7. staged_rollout_endpoints

Wire types: `handlers_rollout.go`. Engine: the `ota-rollout-engine` brick ([github.com/ HelixDevelopment/ota-rollout-engine](https://github.com/HelixDevelopment/ota-rollout-engine)), driven via `server/internal/rollout.Service` (`s.rollout`, wired in `server.go:89`). Enum values below are taken from the brick source (`types.go`, `verdict.go`).

7.1 POST `/api/v1/deployments/{deploymentId}/rollout`

Create + start a staged rollout for an existing deployment.

- **Roles:** operator, admin (`server.go:140`). **Maps to:** `handleCreateRollout`.
- **Request body** (`RolloutCreate`): `{ "phases": [RolloutPhaseSpec, ...] }`, **at least one** phase (empty → 400 `VALIDATION_FAILED`, `details: [{field:"phases",issue:"must not be empty"}]`). Each `RolloutPhaseSpec`:

```
{
  "percentage": 10,
  "success_threshold": 0.95,
  "error_threshold": 0.02,
  "duration_seconds": 3600,
  "auto_progress": true
}
```

`duration_seconds` is converted to a `time.Duration` for the engine

`Phase.Duration`. - **Pre-check:** the deployment must exist

(`s.repo.GetDeployment`) → else 404 `NOT_FOUND` ("deployment not found"). -

Response 201 (`RolloutState`):

```
{
  "deployment_id": "d12b...",
  "status": "active",
  "current_phase": 0,
  "phases": [ { "percentage": 10, "success_threshold": 0.95,
               "error_threshold": 0.02, "duration_seconds": 3600,
               "auto_progreess": true } ],
  "updated_at": "2026-06-08T10:30:00Z"
}
```

- **status** is the engine `Status` string. Closed set (brick `types.go`): `pending`, `active`, `halted`, `completed`, `held`.
- **Status codes:** 201 Created; 404 `NOT_FOUND` (deployment missing); 400 `VALIDATION_FAILED` (malformed body, empty phases, or the brick's plan validation failing — the handler maps a `CreateAndStart` error to 400 with the brick error string in `details`, e.g. percentages must be strictly increasing and end at 100, thresholds in `[0, 1]`).

7.2 GET `/api/v1/deployments/{deploymentId}/rollout`

- **Roles:** viewer, operator, admin (`server.go:141`). **Maps to:** `handleGetRollout`.
- **Response 200** (`RolloutState`, same shape as §7.1).

- **Status codes:** 200 OK; 404 NOT_FOUND ("no rollout for this deployment" — when Service.Get returns the engine's not-found).

7.3 POST /api/v1/deployments/{deploymentId}/rollout/evaluate

Apply a telemetry-derived health verdict to the current phase and return the engine decision.

- **Roles:** operator, admin (server.go:142). **Maps to:** handleEvaluateRollout.
- **Request body** (RolloutVerdict):

```
{ "success_rate": 0.97, "error_rate": 0.01,
  "post_boot_health_failed": false }
```

- **Response 200** (RolloutDecision):

```
{
  "action": "advance",
  "reason": "success_threshold_met",
  "state": { "deployment_id": "d12b...", "status": "active",
    "current_phase": 1, "phases": [], "updated_at": "..."}
}
```

- **action** — engine Action (brick verdict.go): halt, advance, hold, complete.
- **reason** — engine Reason (brick verdict.go): error_threshold_breached, post_boot_health_failed, success_threshold_met, evaluation_window_open, window_expired_below_threshold, auto_progress_disabled, no_active_phase.
- **state** — the post-evaluation RolloutState (re-fetched via Service.Get).
- **Status codes:** 200 OK; 404 NOT_FOUND (engine.ErrNotFound → "no rollout for this deployment"); 400 VALIDATION_FAILED (malformed verdict body, or a non-not-found evaluate error → "could not evaluate rollout").

8. planned_recall_endpoint_not_yet_wired

The operator **recall** (server-driven rollback to N-1) surface from ../1.0.1-staged-rollout/rollback_ux.md §7 is **PLANNED / not-yet-wired**. It is documented here only to record that it is *absent* from the built route table:

- POST /api/v1/deployments/{deploymentId}/recall — **not present** in server.go.
- GET /api/v1/deployments/{deploymentId}/rollbacks — **not present** in server.go.

Per rollback_ux.md §7 and §11, these depend on the migration-002 pgx StoragePort over the rollback_history table (phase README “REMAINING”), and **no rollout.Service rollback/recall method exists today**. The route group these would extend (/deployments/:deploymentId/rollout*) is real (§7); the recall routes are not. This section will be promoted to a live endpoint section when the handler + route land.

9. audit_write_middleware_cross_cutting

The audit **write** path is not an endpoint but is the source of every GET /audit row, so it is recorded here for completeness (handlers_audit.go:auditMiddleware, wired on the protected group at server.go:123).

- **Placement (load-bearing):** mounted on the auth group **with** authMiddleware and **after** requireRole on each route, and writes **after** the handler (c.Next()) then inspect c.Writer.Status()). So an RBAC-rejected (401/403) request is never audited, and only a 2xx mutating action is logged.
- **What is audited:** mutating methods only (POST/PUT/PATCH/DELETE, via isMutating) that returned 2xx. GETs and failed mutations are not written. GET /audit is itself not audited (it is a GET).
- **Action verb** (deriveAuditAction): <RESOURCE>_<VERB> SCREAMING_SNAKE_CASE derived from the gin route template (never the raw path, so ids never leak), e.g. GROUP_CREATE, GROUP_UPDATE, GROUP_DELETE, DEVICE_REGISTER, ARTIFACT_UPLOAD, and GROUP_MEMBER_* for the members sub-routes.
- **Best-effort:** AppendAudit errors are swallowed (_ =) — a failing audit sink never fails the user's already-successful request.

10. spec_vs_implementation_divergences

Per Constitution §11.4.6, where the prose specs and the built code disagree, the **code is authoritative** and the difference is stated here, never silently reconciled.

Audit (operational_endpoints.md §4 vs handlers_audit.go):

- The spec's actor is a nested object { user_id, subject }; the built AuditLogEntry.actor is a **flat string** (subject, falling back to user id).
- The spec lists query params ?actor, ?resource_id, ?since, ?until; the built handler consumes only ?action, ?resource_type, ?cursor, ?limit. There is **no** time-window or actor/resource-id filtering in the implementation.
- The spec's response model is AuditList; the built type is AuditLogList.

Telemetry (operational_endpoints.md §5 vs handlers_telemetry.go):

- GET /devices/{id}/telemetry: the spec specifies pagination (?limit/?cursor), filters (?event_type, ?since, ?until, ?deployment_id), newest-first ordering, and an items array with id/success/duration_ms/bytes_transferred. The built handler takes **no query params**, returns the full history in **insertion order** under an **events** key, and the per-event view is {event, version, deployment_id, error_code, detail, timestamp, received_at} — no id, success, duration_ms, or bytes_transferred. It does **not** emit 404 for an unknown device (an empty history is returned).
- GET /telemetry/overview: the spec specifies a rich aggregate (scope, devices_total, devices_reporting, by_state latest-per-device, event_counts, failure_rate, generated_at) plus ?deployment_id/?os/?since/?until scoping. The built response is the minimal { "event_counts": {type:int64}, "total": int64 } with **no** scoping, **no** per-device state breakdown, and **no** failure_rate.

Groups (operational_endpoints.md §6 vs handlers_group.go):

- The built GroupView is { id, name, description, created_at } — there is **no** filter_criteria and **no** member_count (the spec's Group carries both). The response key for the id is id, not the spec's group_id.
- GET /groups has **no pagination** (no ?name/?limit/?cursor); the spec specifies a paginated GroupList.
- POST /groups/{id}/members accepts a **single** { "device_id": "... " } and returns 204; the spec specifies a **batch** { "device_ids": [...] } body returning 200 with a {added, already_member, not_found} result. There is no partial-success reporting in the build.
- GET /groups/{id}/members returns { group_id, device_ids: [string] }; the spec specifies an items array of { device_id, added_at } objects with pagination.
- DELETE /groups/{id}/members/{deviceId} requires operator/admin in both spec and code (consistent). DELETE /groups/{id} requires admin in both (consistent).

Roles — consistent: the role assignments in the route table match operational_endpoints.md §3 for every operational route, and the rollout role assignments match rollout_engine.md §8 (create/evaluate operator/admin; get viewer+).

Rollout — consistent with the brick: RolloutState.status and RolloutDecision.action/ reason enumerate the exact ota-rollout-engine Status/Action/Reason string values; no values are invented. The handler maps deployment-missing → 404, plan-invalid → 400, and evaluate-not-found → 404.